Contents

Threat Register **1**
Status . . . 1
Legend . . . 1
Register . . . 1
Explicitly out of scope . . . 19
Maintenance . . . 19
Related . . . 20

Threat Register

Subject V.6 requires more than protections: *"you must identify other dangers and describe the protection measures that can be implemented"*. This is that document.

A written accepted risk is an engineering decision. An unwritten one is an oversight. Evaluators ask what you did *not* protect against; this table is the answer.

Status

Design-stage register. 29 threats, plus one accepted residual risk recorded in the Accepted section (T-30, the hub's query-string token). Slice S0 landed the first verified rows, S1 added T-25, T-26 and T-27, and closed T-25; S5 closed T-06 — the reorder race subject V.2.3 grades — gave T-11 and T-12 their first evidence, and accepted T-30. S6 filled T-17: the location-and-time licence is enforced, and its residual risk is restated rather than quietly closed. Every other mitigation below is *designed*, not verified. The Verified column stays empty until a test, a config artifact or a named code path proves it.

Legend

Likelihood and Impact: L / M / H. Residual = what remains **after** the planned mitigation.

Register

#	Threat	L	I	Planned mitigation	Residual	Accepted by	Verified
T-01	Credential stuffing / brute force on login	H	H	rate limit per account **and** per IP, progressive backoff, generic errors, BCrypt cost >= 12	**mitigated 2026-09-27 by SPEC-006**, within the limits of one instance: distributed low-and-slow attempts across many IPs remain possible, the per-account counter is the control that bounds them, and there is no breached-password feed at launch. A restart clears every counter	owner, 2026-09-27, on SPEC-006	S1: `CredentialRateLimitingT` `CredentialAttemptLimite` and `BCryptPasswordHasherTes` for the cost floor
T-02	Session hijacking via stolen access token	M	H	15-minute access token, HTTPS only, no token in URLs or logs	a stolen token is valid until expiry; no device binding		—
T-03	Refresh-token theft	M	H	opaque, hashed at rest, rotated on every use, family revoked on reuse detection	detection is after the fact, not preventive		—

#	Threat	L	I	Planned mitigation	Residual	Accepted by	Verified
T-04	Broken object-level authorization (another user's data)	M	H	two gates in a pipeline behaviour, checked on every request and hub join; hidden resources return `404`	a new endpoint that bypasses the behaviour; mitigated by a required negative test per endpoint		—
T-05	Vote manipulation (double voting, stuffing)	H	M	unique index `(EventId, TrackId, UserId)` as the arbiter, rate limit on the vote endpoint	multiple accounts by one person (sybil) is not prevented; email verification raises the cost only		—
T-06	Playlist reorder conflict causing lost or divergent order	H	M	a `bigint` sparse ordering key, and every track write updating the playlist row inside its own transaction keyed on the `xmin` the caller read — so the guarded `UPDATE` is both the precondition and the lock that serialises writers	rapid concurrent edits still surface as `412` to the user, who must reload and reapply. The status is `412` rather than `409`: the precondition in the request is what failed, and the fix is a reload either way	**closed by SPEC-015 on 2026-10-01**	S5: `PlaylistTrackConcurrenc` — twenty parallel moves of one track with one token give exactly one `200`, nineteen `412`, and a final order that is a permutation of the original with distinct positions

#	Threat	L	I	Planned mitigation	Residual	Accepted by	Verified
T-07	Account takeover through unverified provider email matching	L	H	match an existing account only when the provider asserts `email_verified`	a provider that wrongly asserts verification		—
T-08	ID token substitution (token minted for another app)	L	H	validate `aud`, `iss`, `exp`, `nonce` against JWKS, server-side	depends on correct JWKS caching and key rotation handling		—
T-09	API key or secret committed to git	L	**critical**	`.env` git-ignored from commit 1, `.env.example` placeholders only, rotate-first leak procedure	subject V.8 makes this an **automatic project failure**; process discipline is the only control		S0: `MusicRoom/.dockerignore` excludes `.env`; no `.env` and no credential string in any layer of the built image; `docker-compose.yml` carries no literal

#	Threat	L	I	Planned mitigation	Residual	Accepted by	Verified
T-10	YouTube quota exhaustion during the demo	M	M	**built 2026-10-04 by SPEC-025.** `videos.list` is preferred over `search.list`; both cache levels of [[ADR-0023-catalog-cache]] stand in front of the provider, so a restart does not re-burn the quota and a repeated search re-issues nothing; `GET /tracks` — the only route that can reach `search.list` — carries a per-account limit of 30 a minute; every call logs its unit cost so the day's budget is measurable before it runs out; `403 quotaExceeded` answers `503` with its own problem type, and a stale	**residual, carried deliberately.** No per-account quota *budget*: the limit bounds the rate, not the day's total, so thirty accounts searching all day can still exhaust a free tier. Nothing alerts when the budget runs low — the cost is in the log and nothing reads it. And the client-side debounce `youtube-integration` asks for does not exist yet, because `MusicRoomApp/` is empty until after S9; until then one keystroke per request is possible from a hand-written client, bounded only by the rate limit		S4: `YouTubeCatalogProviderT` — the quota cost is in the log line, a `403` becomes an unavailable outcome rather than a `500`, and a provider `5xx` is retried a bounded number of times; `CachedCatalogTests` — a second read of one video and a repeated search both reach the provider zero times, a fresh database row calls nothing at all, and a provider outage serves the stale row; `TrackEndpointTests` — `503` on both routes when the catalog cannot answer, and `422` for a blank query so no quota is spent on it. No suite calls YouTube

#	Threat	L	I	Planned mitigation	Residual	Accepted by	Verified
T-11	Denial of service via unbounded input or list requests	M	M	capped `pageSize`, bounded string and collection lengths, global plus per-endpoint rate limits, and a hard **500-track cap per playlist** answered `409` (SPEC-015), which also bounds the renumber a sparse key occasionally needs	a single instance has no upstream protection; capacity is whatever V.7 measures. The cap is per playlist, not per account: an account may still hold many playlists		S5: `PlaylistTrackEndpointTe`

#	Threat	L	I	Planned mitigation	Residual	Accepted by	Verified
T-12	SignalR connection exhaustion	M	M	bounded payloads, and explicit `KeepAliveInterval` of 15 seconds with a `ClientTimeoutInterval` of 60 (SPEC-016) rather than the framework defaults, so a dead mobile connection is dropped in a minute instead of being held; every hub connection must present a valid token and name a playlist it may see before it joins anything	no backplane (issue O8), so one instance is the ceiling. Nothing caps connections **per account** today: a client that reconnects in a loop is bounded by the rate limiter on the REST side and by nothing on the hub		S5: the two settings are a named code path in `RealtimeServices`; `PlaylistHubTests` proves an anonymous handshake is refused
T-13	Database connection-pool exhaustion	M	M	small pool, short-lived connections, pooled connection string for the API	Supabase enforces a hard connection ceiling above ours		—

#	Threat	L	I	Planned mitigation	Residual	Accepted by	Verified
T-14	Personal data over-exposure in profile responses	M	M	per-field filtering server-side by requester relation-ship	a new profile field added without a visibility tier; mitigated by making the tier non-nullable		—
T-15	Sensitive data in logs	M	H	never-logged list, structured templates, no full bodies on credential endpoints	a developer logging an object graph that contains a token		partial, S0: `GetHealthReady_WhenData` proves the readiness body carries no host, connection string or exception text
T-16	Vulnerable transitive depen-dency	M	M	`dotnet list package --vulnerable --include-transitive` fails CI on high severity	a zero-day between scans		S0: step present in both workflows and in `make agent-check`; clean across all 10 projects

#	Threat	L	I	Planned mitigation	Residual	Accepted by	Verified
T-17	Location+time licence bypass (spoofed GPS)	M	M	a server-side great-circle check against the event's own centre and radius, **plus** an optional short-lived venue code — stored as a SHA-256 digest, 60 to 3600 seconds, single-use per account, rate limited per account — with the event's policy selecting `Geofence`, `EventCode` or `Both`; every evaluation uses server time, and a `LocationAndTimeRestricted` event cannot exist without a configured proof (SPEC-017 decision 3) — [[ADR-0014-location-time-licence]]	**client-reported coordinates remain spoofable**, and a code can be relayed by text to somebody off-site. Neither is fixable without infrastructure this project will not build, and both are stated plainly at defence: subject V.6 does not expect an unbreakable API, it expects the remaining danger to be `TimeRestricted`. What the design does buy is cost — a remote voter needs a confederate at the venue, per code, per account	**accepted, owner, 2026-09-24 on ADR-0014; the implementation of 2026-10-01 changes nothing about the residual risk**	S6: `EventPresenceEndpointTe` proves a coordinate outside the radius, a vote outside the window, an unknown code, an expired code, a code reused by the same account and a `Both` policy with only a code are each refused `422`, and that the same code admits a second account once; `EventVoteConcurrencyTes` proves the single use holds under concurrency; `GeodesyTests` proves the distance, including across the antimeridian

#	Threat	L	I	Planned mitigation	Residual	Accepted by	Verified
T-18	Malicious content in user-supplied text (event names, playlist titles)	M	M	validation, length caps, output encoding at the consumer	the client is responsible for rendering safely; no web frontend today		—

#	Threat	L	I	Planned mitigation	Residual	Accepted by	Verified
T-19	Mail provider quota exhausted by a registration loop	L	M	rate limits on registration and verification resend; hosted provider free tier — [[ADR-0013-mail-provider]]	a quota exhausted mid-defence blocks the verification demo. **The rate limiter is a precondition of slice S1a** (owner decision, SPEC-004, 2026-09-27): `POST /api/v1/auth/verification-requests` is the first endpoint whose success path is one mail send per anonymous request, so the moment O17 lands the hosted provider the loop stops being theoretical. While `LoggingMailService` is the only sender the cost is log lines on a Development host, which the SPEC-003 start-up guard confines to that environment		—

#	Threat	L	I	Planned mitigation	Residual	Accepted by	Verified
T-20	Supabase free project pauses from inactivity	M	H	touch the project before the defence; local PostgreSQL container as a fallback with identical migrations — [[ADR-0016-supabase-plan-region]]	a pause discovered on the day still costs demo time		—
T-22	Malicious controller skips the queue	L	M	only `controllerUserId` may advance; `expectedNowPlayingId` makes a replay idempotent; the owner can revoke control at any time	a legitimate controller can still play ahead — that is what control means		—
T-23	Playback stalls because the controller device went offline	M	M	control takeover via `PUT /devices/{id}/control`; the owner can revoke	no server-side watchdog by design ([[ADR-0017-playback-session]]); recovery is manual (issue O13)		—

#	Threat	L	I	Planned mitigation	Residual	Accepted by	Verified
T-24	Operational endpoints reachable by anyone who can reach the port	M	L	`/health` and `/health/ready` are anonymous by contract ([[endpoint-inventory]]) and report only check names and states — no version, host, connection string or exception text	an unauthenticated caller learns the service is up and whether its database is reachable, and can poll it: the endpoints carry no rate-limit policy, because a limiter that throttles a probe defeats the probe		S0: readiness body asserted by test; no authentication is planned for these two

#	Threat	L	I	Planned mitigation	Residual	Accepted by	Verified
T-25	Anonymous auth endpoints have no rate limit: `POST /api/v1/auth/registrations`, `POST /api/v1/auth/email-verifications`, `POST /api/v1/auth/verification-requests`, `POST /api/v1/auth/sessions` and `POST /api/v1/auth/tokens` can all be driven in a loop. SPEC-007 added `POST /api/v1/auth/password-reset-requests` and `POST /api/v1/auth/password-resets`, both limited from their first commit	M	M	rate limits keyed on IP and on the submitted address (SDD §13.3), built once for registration, verification, login and resend together	**closed 2026-09-27 by SPEC-006.** Deferred three times while the control was shared with endpoints that did not exist; built once, for all five at once. Each endpoint carries its own per-IP policy, so a registration loop cannot spend the sign-in allowance, and login, registration and resend also carry a per-account counter with progressive backoff. Refusals are `429` with `Retry-After` and an empty body, so the refusal itself says nothing about which key tripped. **Residual, carried deliberately:** limiter	owner, 2026-09-27, on SPEC-006	S1: `CredentialRateLimitingT` — per-IP limit with `Retry-After`, per-account limit scoped to one address, a second client unaffected, and the registration, resend, verification and rotation loops each bounded; `CredentialAttemptLimite` — the window, the doubling penalty, the ceiling and the reset on success

#	Threat	L	I	Planned mitigation	Residual	Accepted by	Verified
T-26	Development-only `LoggingMailService` writes the email verification link **and the password reset token** to the application log	M	M	registered as `IMailService` only when the host environment is Development; `AddInfrastructureServices` throws `InvalidOperationException` at start-up in any other environment rather than falling back silently or booting with no sender. With no `VERIFICATION_LINK_BASE_URL` configured the sender logs the bare token and says the variable is missing — it never invents a default link	**accepted, and widened by SPEC-007 on 2026-09-27.** Anyone who can read a development container's logs can verify an account created on that machine — and, since the reset token is logged too, can **take that account over**: the token replaces its password. That is a larger exposure than verification and the owner accepted it knowingly, because the alternative was shipping the recovery flow untested against a real client. The exposure is bounded to Development hosts by a mechanical guard, not	owner, 2026-09-26, on SPEC-003	S1: `LoggingMailServiceTests` asserts one event carrying the link, one warning-level event carrying the reset token and naming this row, and the non-Development start-up throw is a named code path in `InfrastructureServiceRe`

#	Threat	L	I	Planned mitigation	Residual	Accepted by	Verified
T-27	No security event is logged when a credential endpoint refuses a request: a failed login, a rejected refresh token and a detected refresh-token **reuse** all leave no record beyond the Serilog request line	M	M	the A09 event list in [[owasp-top10-checklist]]: log failed login, lockout, token reuse, authorisation denial and rate-limit trip, through the audit behaviour of slice S2 ([[ADR-0019-hub-action-auditing]], SDD §13.4)	**closed 2026-09-28 by SPEC-009.** Every credential refusal now leaves an `audit_records` row that says which refusal it was: `refused` for a wrong password or an unknown token, `throttled` when the account is out of attempts, `expired` for a spent window, and `reusedetected` when an already-rotated refresh token revoked a family — that last one naming the account whose family died. Recording a throttled attempt required the limiter's refusal to stop being an exception, because	owner, 2026-09-28, on SPEC-009	S2: `AuditRecordEndpointTest` — refused, completed with a named account, `reusedetected`, expired, and a duplicate registration staying `completed`; `CredentialRateLimitingT` — a `throttled` row with the `429` unchanged; `AuditBehaviourTests` — every outcome value and the silent default

#	Threat	L	I	Planned mitigation	Residual	Accepted by	Verified
T-28	Neither a password reset nor a logout revokes access tokens already issued: a JWT minted before either stays valid until `exp`, so a thief who signed in moments earlier keeps API access for the remainder of that window	M	M	the 15-minute access-token lifetime (SDD §13.1), which is why the lifetime is short; a `jti` or per-user not-valid-before denylist checked on every authenti-cated request would close it entirely	**accepted.** Revoking refresh families ends every session's ability to continue, which is the control the contract promises; the remaining exposure is bounded by the token lifetime and cannot exceed **15 minutes and 30 seconds** — 15 minutes signed plus the 30 seconds of clock skew the bearer handler allows (owner decision, 2026-09-28; the handler's five-minute default would have made it 20 minutes). SPEC-010 widened this row from reset to logout as well, for the same reason: a stateless token cannot be recalled without	owner, 2026-09-27, on SPEC-007	S1: `Reset_WithTheIssuedToke` proves what a reset **does** revoke; S2: `Logout_ThenRotatingTheR` proves the same for logout, and `BearerValidationTests.S` proves the window closes. The window itself is stated, not measured

#	Threat	L	I	Planned mitigation	Residual	Accepted by	Verified
T-29	The audit log is fail closed: a command whose `audit_records` insert fails is rolled back and answers `500`, so a fault confined to the audit table stops every mutating endpoint	L	H	the record and the effect share one transaction, so neither can exist without the other (SDD §13.4, §10.7); the table has two indexes and no foreign key, so the write is as cheap as an insert gets	**accepted, deliberately.** The alternative was fail open, which allows an action nobody can account for — the exact gap subject V.6 asks about. The owner chose evidence over availability on 2026-09-27, knowing an audit-table fault becomes a total mutation outage and that a defence demo would die on it. Reads, health probes and the OpenAPI document are unaffected, so the service stays diagnosable while refusing to act unrecorded. **SPEC-009 widened what this**	owner, 2026-09-27, on SPEC-008	S2: `AuditBehaviourTests.Han` proves the rollback and the rethrow

#	Threat	L	I	Planned mitigation	Residual	Accepted by	Verified
T-21	Voter location data stored unnecessarily	L	M	the submitted coordinate is used for the check and **not** persisted on the vote row; event centre stored at coarse precision	the coordinate exists in transit and in request logs unless excluded — add it to the never-logged list		—

Explicitly out of scope

Not defended against	Why
A determined attacker with unlimited resources	subject V.6: *"your API is not expected to be unbreakable"*
Physical device compromise	the token is in the platform secure store; a rooted device is outside our control
Supabase platform compromise	vendor trust boundary, stated rather than pretended away
Sybil accounts (T-05)	email verification raises cost; full prevention needs identity proofing this project will not build
An access token in a hub URL's query string (T-30)	A WebSocket handshake cannot carry an `Authorization` header, so SignalR documents `access_token` as a query parameter and the mobile client has no other option. Bounded three ways: it is read **only** under `/hubs`, so no REST route gains a second way to authenticate; Serilog's request logging records the path and not the query string, so it does not reach the application log; and the token already lives 15 minutes (ADR-0030, ADR-0032). What remains is that a URL is more copyable than a header — in a crash report, a proxy log the deployment owns, or a shared screen. Accepted on the same ground as the rest of the transport story: TLS is the deployment's job, and the alternative is no realtime on mobile at all (SPEC-016 decision 4)

Maintenance

- A finding from [[owasp-top10-checklist]] or [[owasp-api-top10-checklist]] that is not fixed in the pull request gets a row here **before** that PR merges.
- Fill Accepted by with a real name when the team agrees to carry a residual risk.
- Fill Verified with the test or artifact that proves the mitigation, once it exists.
- Never delete a row. Close it by pointing at the evidence.

Related

[[owasp-index]] · [[asvs-conformance]] · [[application-security]] · [[authentication-authorization]] · [[sdd]] §13.6 · [[subject-requirements]]